

Input & Output Statements

1. What is the purpose of input/output functions in C programming

- a. To perform arithmetic operations
- b. To manage memory allocation
- c. To facilitate interaction between the user and the program
- d. To define the structure of a program

Explanation: Input/output functions are used to interact with the user by reading input and displaying output.

2. Which library is required to use I/O functions in C

- a. <stdlib.h>
- b. <conio.h>
- c. <stdio.h>
- d. <string.h>

Explanation: The <stdio.h> library provides declarations for input/output functions like printf() and scanf().

3. What is the role of the printf() function in C

- a. To take input from the user
- b. To print output to the console
- c. To declare variables
- d. To execute loops

Explanation: The printf() function is used to display formatted output on the console.

4. What is the purpose of the scanf() function in C

- a. To read data from the user
- b. To display output
- c. To perform mathematical calculations
- d. To allocate memory

Explanation: The scanf() function reads formatted input from the user.

5. What is the difference between formatted and unformatted I/O functions

- a. Formatted I/O uses headers, unformatted does not
- b. Formatted I/O processes structured data, unformatted handles raw data**
- c. Formatted I/O works faster than unformatted
- d. Unformatted I/O uses special characters

Explanation: Formatted I/O functions process structured data, while unformatted I/O functions deal with raw data.

6. What is a format specifier in C programming

- a. A variable declaration tool
- b. A code for representing data types in I/O functions**
- c. A function to manage memory
- d. A way to write conditional statements

Explanation: Format specifiers are used in I/O functions to specify the type of data being read or displayed.

7. Which format specifier is used for integers in C

- a. %d**
- b. %f
- c. %c
- d. %s

Explanation: The %d format specifier is used to represent integers in input/output operations.

8. What is the purpose of the address operator in scanf()

- a. To allocate memory dynamically
- b. To provide the memory address of a variable**
- c. To specify the type of data
- d. To read characters from the user

Explanation: The address operator & is used to pass the memory address of a variable to scanf().

9. What is a precision specifier in C

- a. A way to allocate memory
- b. A method for precise calculations
- c. A control for output formatting**
- d. A specifier for handling errors

Explanation: Precision specifiers in C control the formatting of output, such as the number of decimal places.

10. What happens if the wrong format specifier is used in printf()

- a. The program crashes
- b. The output is incorrect or undefined**
- c. The compiler skips the line
- d. The function terminates execution

Explanation: Using the wrong format specifier in printf() can result in incorrect or undefined output.

11. What is an example of an invalid format specifier

- a. %z**
- b. %d
- c. %f
- d. %c

Explanation: %z is not a valid format specifier in C, unlike %d, %f, and %c.

12. How does the %s specifier handle strings in printf()

- a. Prints the string with a newline
- b. Prints the string until a null character is encountered**
- c. Trims spaces from the string
- d. Reverses the string

Explanation: %s prints characters in the string until a null character (\0) is encountered.

13. Which format specifier is used to print floating-point numbers in C

- a. %c
- b. %d
- c. %s
- d. %f**

Explanation: The %f format specifier is used for printing floating-point numbers in C.

14. What does the %c format specifier represent

- a. A string
- b. A single character**
- c. An integer
- d. A floating-point number

Explanation: The %c format specifier is used to represent a single character in C.

15. How can we control the number of decimal places in printf()

- a. By using a format specifier like %.nf
- b. By specifying precision in scanf()
- c. By adding additional parameters in printf()
- d. By using the address operator

Explanation: To control decimal places in printf(), we use precision specifiers like %.2f.

16. Which format specifier is used to display hexadecimal numbers

- a. %x
- b. %d
- c. %f
- d. %c

Explanation: The %x format specifier is used to display hexadecimal numbers in C.

17. What is the output of the following C code?

```
#include <stdio.h>

int main() {
    float num = 3.14159;
    printf("%.2f", num);
    return 0;
}
```

- a. 3.14
- b. 3.14159
- c. 3.1
- d. 3

Explanation: The precision specifier %.2f in the printf function limits the number of digits displayed after the decimal point to 2. In this case, the floating-point value 3.14159 is rounded to two decimal places, which results in 3.14. The %.2f specifier doesn't affect the integer part of the number but rounds the fractional part to two digits. Thus, the output is 3.14.

18. Which format specifier is used to display octal numbers

- a. `%o`
- b. `%d`
- c. `%f`
- d. `%c`

Explanation: The `%x` format specifier is used to display hexadecimal numbers in C.

19. What is the correct syntax for printing an integer using `printf`?

- a. `printf("%d", num);`
- b. `printf("%i", num);`
- c. `printf("%f", num);`
- d. `printf("%s", num);`

Explanation: The `%d` format specifier is used to print an integer in C. The `printf` function formats the output based on the format specifier provided, and `%d` is specifically for integers.

20. How would you read an integer value from the user using `scanf`?

- a. `scanf("%i", &num);`
- b. `scanf("%d", num);`
- c. `scanf("%d", &num);`
- d. `scanf("%f", &num);`

Explanation: `scanf` requires the address of the variable to store the input value. The `%d` format specifier is used to read integers, and the `&` operator is needed to pass the address of the variable `num`.